esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Programmation système

LAB4 - La gestion des processus

Table des matières

Objectifs 2
Rappels et mise en place de l'environnement 2
Gestion des processus 2
1Shell 2
2Création de processus en C 3
3Schéma « gestion des processus » 4

By R

Objectifs

Ce « LAB » présente les notions de base de gestions de processus. Il est basé sur la mise en parallèle de notions relatives aux commandes Shell (et Scripts Shell) avec leur équivalent en langage C.

- Documents : «Linux : la gestion des processus ».
- Notions abordées : le « scheduler », les signaux, la gestion synchrone et asynchrone, /proc
- Commandes et fichiers exploités : ps, pstree, top, trap, kill, fork(), exec().
- **Travail à rendre :** Vous devrez répondre directement à plusieurs questions au sein de ce document, vous le déposerez sur Moodle sous le nom : ***LAB4_noms.pdf***. Il y a un code C à écrire dans la question 2, vous le déposerez sur Moodle sous le nom ***part-2.c***.

Gestion des processus

.1 Shell

- Dans un terminal : lancez un éditeur de texte simple (*gedit*, *Vim*, *Emacs*, etc.). En se lançant, votre éditeur affiche (ou pas...) des informations sur le canal des erreurs. Comment feriez-vous pour qu'il n'affiche plus ces erreurs ? gedit new_file 2> /dev/null. Testez. Info : il est toujours intéressant de lancer les applications graphiques à partir d'un terminal, car cela permet de voir des dysfonctionnements souvent cachés par les interfaces graphiques (idem sous Windows).
- Pourquoi n'avez-vous plus la main sur votre terminal ? Par ce que le processus pere(shell) s'est mis en attente(wait). Lancez en aveugle la commande « id ». Fermez votre éditeur via ses menus graphiques. Cela est-il normal ? Oui c'est normal, par ce que le processus pere(shell) à repris la main. Relancez-le de manière asynchrone et sans affichage des messages d'erreur **commande :** gedit new_file 2> /dev/null &
- Définissez les 13 champs affichés par la commande « ps -faxl »

- **« F » :** Est l'indicateur associé au processus

1 engendré mais pas exécuté ; 4 a utilisé des privilèges de superutilisateur ;

- **UID : User Identifier**
- **PID : Numéro d'identification du processus,**
- **PPID : Numéro d'identification du processus du processus parent,**
- **PRI : Identi**ficateur unique du processus qui à été définit par le système,
- **NI : Valeur de politesse, comprise entre 19 (la plus polie) à -20 (la moins polie pour les autres), une valeur négative signifie un accès moindre au cpu**
- **VSZ » :** Taille de la mémoire virtuelle du processus en KIB. (unit"s de 1024 octets)
- **RSS » :** Taille de mémoire résidente (« resident set size » ou RSS), la mémoire physique non transférée vers l'espace d'échange (swap) qu'une tâche a utilisée (en kilooctets).
- **WCHAN » : Nom de la fonction du noyau où le processus est en sommeil, affiche « - » si le processus s'exécute, « * » si le processus possède plusieurs processus légers (« multi-thread ») et ps n'affiche pas les processus légers.**
- **STAT » :** L'état du processus au moment du lancement de la commande
- **TTY » : L**e terminal depuis lequel le processus a été lancé
- **TIME » : L**e temps d 'exécution de la commande par le processeur, en millisecondes
- **COMMAND » : L**e libellé de la commande lancée

- Quels sont le PID et le PPID de votre éditeur ?

```
4     0     4757     1  20   0  71672 16088 -      S    ?          0:00 python3
4     0     4892     1  20   0 248508  6200 -      Ssl  ?          0:00 /usr/li
4     0     5086     1  20   0 1022536 19584 -     Ssl  ?          0:36 /usr/li
```

Le PID de mon éditeur de Texte, Python3 est 4757 et son PPID est de 1

- Quel est le processus PPID ?
- **C'est le processus parent du PID**
- Affichez la liste des signaux disponibles sur votre système **Kill -l**

```
parallels@parallels-Parallels-Virtual-Platform:~$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
parallels@parallels-Parallels-Virtual-Platform:~$
```

-
- En tant que père de votre éditeur, votre Shell peut lui envoyer des signaux. Fermer votre éditeur en lui envoyant le signal d'interruption « SIGINT » **commande : Kill** SIGINT **numéro du PID // Kill-s**.
- Le PPID d'un shell correspond au processus qui gère le terminal et les entrées clavier. Quel est-il sur votre système ? /usr/libexec/gnome-terminal_server. Quelle commande permet de connaître les caractéristiques du terminal : stty -a En analysant cette commande, vous pourrez connaître les séquences de touches associées à certains envois de signaux. Quelle séquence pour le signal SIGINT ? **Contrôle + C**. Testez.
- Lancez votre éditeur de manière synchrone (vous n'avez donc plus la main). Lancez-lui le signal de suspension (SIGTSTP) de deux manières différentes **(1 :Contrôle +Z) (2 :KIll Sigstop + (numéro du pid)**. Vous reprenez ainsi la main. Votre éditeur est bien toujours présent, mais comme le processus ne passe plus dans la boucle du microprocesseur, il est devenu inerte (sans activité). Vérifiez. Vous pouvez le réactiver en lui envoyant le signal (SIGCONT) de deux manières différentes **(1 :_bg) (2 :kill –sigstop + PID)**. Testez. Vous pouvez réactiver tous les processus stoppés via la commande « bg » (background). INFO : vous connaissez maintenant un bon moyen de reprendre la main sur votre terminal quand vous lancez un processus graphique.
- Qu'arrivera-t-il à votre éditeur si son père meurt ? **Il cesse de fonctionne à sont tour.** Testez en fermant le terminal. Rouvrez un terminal et lancez-le de manière asynchrone afin qu'il ne dépende plus de son père en cas de mort subite de ce dernier : **commande : nohup gedit new_file &**. Vérifiez en fermant le terminal. Qui est devenu PPID de votre éditeur ? **systemd.**
- Les signaux sont traités par les processus via les standards de programmation. Le « shell » est un processus comme un autre. Ses programmeurs ont prévu que l'utilisateur puisse intercepter les signaux afin de modifier l'action prévue normalement. La commande « trap » qui est interne au « shell » permet ainsi d'intercepter et de reprogrammer l'action des signaux. Ainsi, dans un « shell », la commande « trap 'ls' *nro_signal* » permet d'intercepter le signal numéro '*nro_signal*' quand il est reçu afin de lancer la commande « ls » au lieu du traitement normalement prévu. À quelle séquence de touche correspond le signal « SIGINT » ? Contrôle + C. Reprogrammer l'action liée à ce signal afin de lancer la commande « ps ». Testez. Comment pouvez-vous réinitialiser cette programmation à sa valeur par défaut ? Trap SIGINT
- Créez un « script shell » qui affiche la liste des usagers de votre système. Cette liste est dans le fichier « /etc/passwd » (vous pouvez judicieusement exploiter la commande « cut »). Dans un terminal (et donc un shell), modifiez le comportement du signal numéro 3 afin de lancer votre script. Ouvrez un deuxième terminal. Déterminez le PID du premier terminal et envoyez-lui le signal numéro 3. Faites une capture d'écran des 2 terminaux. Supprimez l'interception du signal numéro 3.

- Quand un processus reçoit le signal numéro 1 (SIGHUP), il se ferme normalement (HUP = Hang UP = raccrocher). À partir du deuxième terminal, lancez le signal numéro 1 sur le premier terminal. Rouvrez un nouveau terminal et interceptez le signal 1 afin qu'il ne se ferme plus lors de la réception de ce signal. Testez. Quel signal faudrait-il envoyer pour fermer un processus qui ne veut plus rien savoir ? **SIGKIL** Qui traite en fait ce signal ? **L'ordonnaceur ou sheduller** INFO : C'est parfois le seul moyen de supprimer un processus planté (boucle infinie par exemple).

Gestion interactive : Sous KDE, GNOME : <CTRL> + <ECHAP>. Sous Windows : <CTRL> + <ALT> + <Suppr>. Sous l'environnement minimaliste LXDE : commande « lxtask ». Dans un terminal texte : commande « top » (puis « h » pour connaître les interactions possibles).

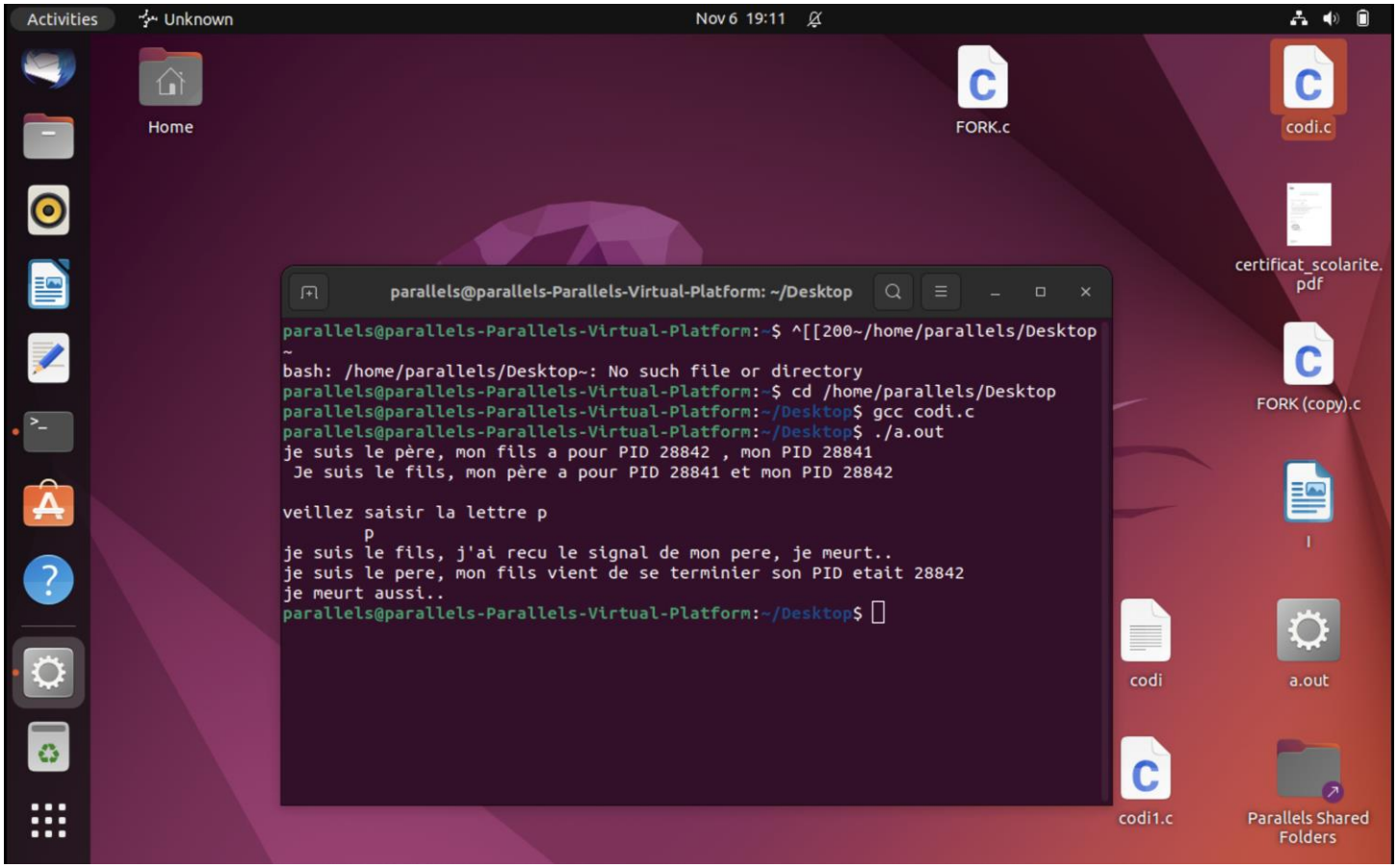
.2 Création de processus en C

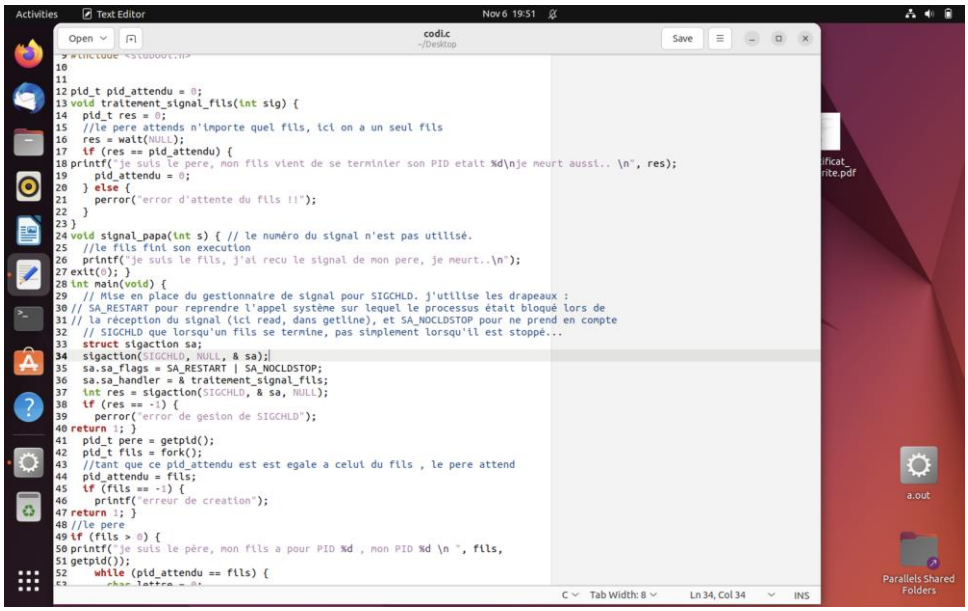
Dans les LAB précédents, vous avez exploité la fonction « system (commande) » de la « librairie C standard (stdlib) » qui lance un « shell » à l'intérieur duquel votre commande est exécutée. Votre programme appelant attend la fin de cet appel pour poursuivre son exécution. L'enchaînement est donc synchrone.
Vous allez maintenant exploiter les possibilités du système multiprocessus qui permet de créer un nouveau processus géré de manière asynchrone et donc traité en parallèle (fonction « fork » et « exec »).
Appuyez-vous sur le guide suivant :

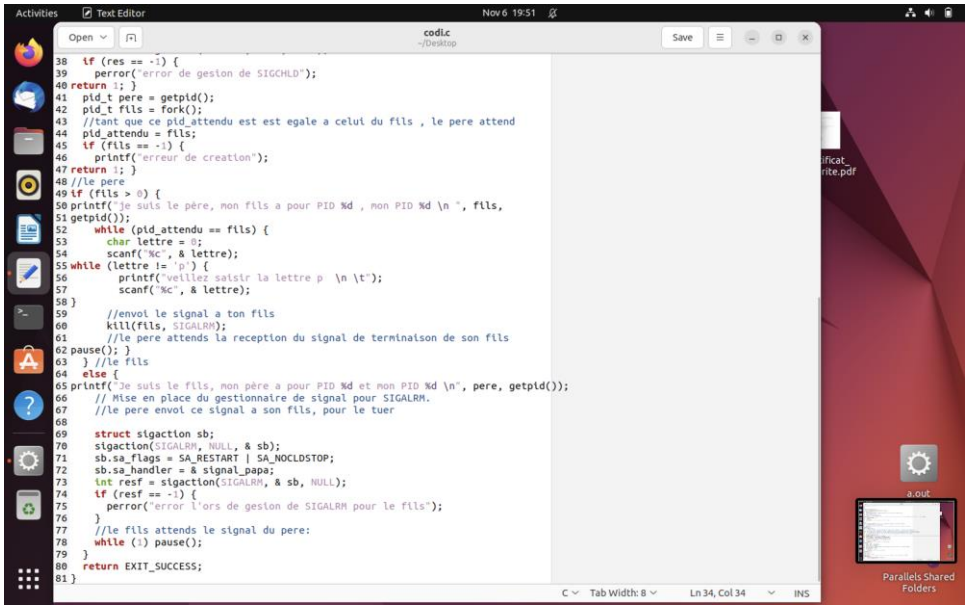
http://mtodorovic.developpez.com/linux/programmation-avancee/?page=page_3#L3

Réalisez un programme en C qui crée un processus fils. Le père affiche son PID et le PID de son fils et attend que vous tapiez la touche « p ». Le processus fils affiche son PID et son PPID. Il ne fait qu'attendre la réception d'un signal de son père (que vous définirez) pour mourir. Le père envoie ce signal quand vous tapez la touche « p ». Quand il s'est assuré que son fils est bien mort, il quitte aussi cette dure vie.

Réalisez une capture d'écran de son exécution.


Activities
Unknown
Nov 6 19:11
Home
FORK.c
codi.c
certificat_scolarite.pdf
FORK (copy).c
I
codi
a.out
codi1.c
Parallels Shared Folders
parallels@parallels-Parallels-Virtual-Platform: ~/Desktop
parallels@parallels-Parallels-Virtual-Platform:~$ ^[[200~/home/parallels/Desktop
~
bash: /home/parallels/Desktop~: No such file or directory
parallels@parallels-Parallels-Virtual-Platform:~$ cd /home/parallels/Desktop
parallels@parallels-Parallels-Virtual-Platform:~/Desktop$ gcc codi.c
parallels@parallels-Parallels-Virtual-Platform:~/Desktop$ ./a.out
je suis le père, mon fils a pour PID 28842 , mon PID 28841
Je suis le fils, mon père a pour PID 28841 et mon PID 28842
veillez saisir la lettre p
p
je suis le fils, j'ai recu le signal de mon pere, je meurt..
je suis le pere, mon fils vient de se terminier son PID etait 28842
je meurt aussi..
parallels@parallels-Parallels-Virtual-Platform:~/Desktop$


Activities
Text Editor
Nov 6 19:51
Open
codi.c
~/Desktop
Save
pid_t pid_attendu = 0;
void traitement_signal_fils(int sig) {
pid_t res = 0;
//le pere attends n'importe quel fils, ici on a un seul fils
res = wait(NULL);
if (res == pid_attendu) {
printf("je suis le pere, mon fils vient de se terminier son PID etait %d\nje meurt aussi.. \n", res);
pid_attendu = 0;
} else {
perror("error d'attente du fils !!");
}
}
void signal_papa(int s) { // le numéro du signal n'est pas utilisé.
//le fils fini son execution
printf("je suis le fils, j'ai recu le signal de mon pere, je meurt..\n");
exit(0); }
int main(void) {
// Mise en place du gestionnaire de signal pour SIGCHLD. j'utilise les drapeaux :
// SA_RESTART pour reprendre l'appel système sur lequel le processus était bloqué lors de
// la réception du signal (ici read, dans getline), et SA_NOCLDSTOP pour ne prend en compte
// SIGCHLD que lorsqu'un fils se termine, pas simplement lorsqu'il est stoppé...
struct sigaction sa;
sigaction(SIGCHLD, NULL, & sa);
sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
sa.sa_handler = & traitement_signal_fils;
int res = sigaction(SIGCHLD, & sa, NULL);
if (res == -1) {
perror("error de gesion de SIGCHLD");
return 1; }
pid_t pere = getpid();
pid_t fils = fork();
//tant que ce pid_attendu est est egale a celui du fils , le pere attend
pid_attendu = fils;
if (fils == -1) {
printf("erreur de creation");
return 1; }
//le pere
if (fils > 0) {
printf("je suis le père, mon fils a pour PID %d , mon PID %d \n ", fils,
getpid());
while (pid_attendu == fils) {
C
Tab Width: 8
Ln 34, Col 34
INS
a.out
Parallels Shared Folders

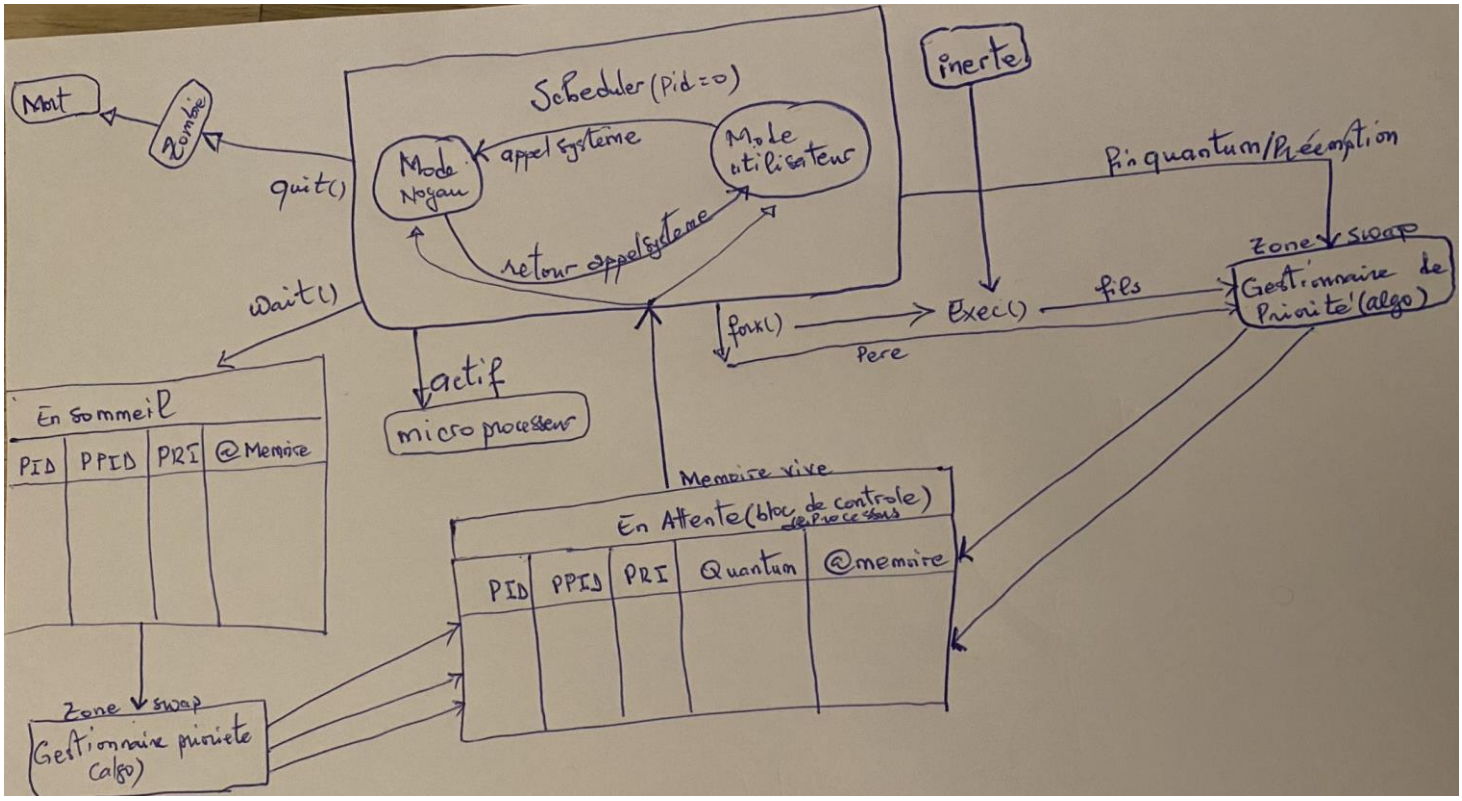


Une fois le fils créé, le père et le fils veulent écrire « je suis le (père/fils) et mon PID est ... » dans un même fichier. Ils ne peuvent cependant pas le faire en même temps... Ajoutez une méthode de résolution de ce problème d'accès concurrent à une ressource (sémaphore, mutex, etc.).

Rendez ce code C sur Moodle dans un fichier part-2.c.

.3 Schéma « gestion des processus »

Réalisez **ci-dessous** le **schéma** de gestion des processus sous Linux. Vous tenterez de faire apparaître les termes suivants : « microprocesseur », « mémoire vive », « zone de swap », « scheduler » « inerte » « en attente », « en sommeil », « actif », « mort », « zombie », « priorité ». Dans un court **texte**, vous expliquerez la vie trépidante d'un processus.



Un processus est un programme en cours d'exécution par un ordinateur. Il est constitué d'un ensemble d'instruction exécuter physiquement par le microprocesseur (Ou CPU)
Tous processus est créé par un processus parent/père, à l'exception du processus init, chargé de l'initialisation du démarrage de la machine.
Le processus créer par l'utilisateur passe dans la zone de swap (Gestionnaire de priorité), et est élu par l'ordonnanceur(sheduler) si sa priorité le permet. Dès lors qu'il est élu, le processus se met au travail pour accomplir sa tache.
Il est possible qu'il ait besoin d'une ressource qui n'est pas disponible dans l'imédiat, et donc le sheduler le met en suspension (ou en attente de ressource). Dès que le ressource est disponible il repasse dans la zone swap et selon sa priorité il est élu par le shelduler.

Lorsqu'un processus a fini son exécution (ou terminer manuellement s'il est en cours d'exécution) il est directement tué et libère ainsi RAM et puissance de calcul. Sinon il passe en mode zombie en attendant d'être élu par le scheduler pour enfin être tué.
La vie d'un processus est en effet trépidante, et même intéressante. J'ai donc regardé certains documents, me rendant compte qu'une petite amélioration peut faire une grande avancé dans la gestion des taches. Mais l'inverse est vrai également. Qui sait, peut etre que sa vie pourrait changer positivement?